```
from bakery import assert_equal

from dataclasses import dataclass

@dataclass

class Compound

    name: str

    mass: float

    molarmass: float

    coeff: int

@dataclass

class Reaction:

    reac1: Compound

    reac2: Compound

    prod1: Compound

    prod2: Compound

#helper function 1: mass to mole

def masstomole(mass: float, molarmass: float) -> float:

    ‘’’

    Purpose: Takes in a mass of element/compound/etc, and calculates the amount it equals in moles of the same element/compound using the molar mass.

Stoich: mass (g)/1(mol (mol)/molarmass (g))=mol

    Args: mass (float) - the amount of stuff there is in grams

molarmass (float) - the amount of grams there are in 1 mol of the compound

    Returns: amountinmoles (float) - the amount of stuff there is in moles

    ‘’’

    amountinmoles = mass / molarmass

    return amountinmoles
```

```
#helper function 2: mole to mole

def moletomole(molesinput: float, coeffinput: int, coeffoutput: int) -> float:

    ‘’’

    Purpose: Takes in the amount of input material in moles, as well as the coefficients of the input and output materials. Completes a calculation to give the amount of output material in moles.

Stoich: molesinput (mol input)/1(coeffoutput (mol output)/coeffinput (mol input)) = mol output

    Args: molesinput (float) - the amount of the input material in moles

coeffinput (int) - the coefficient of the input material from the balanced reaction

coeffoutput (int) - the coefficient of the output material from the balanced reaction

    Returns: molesoutput (float) -

    ‘’’

    molesoutput = molesinput * coeffoutput / coeffinput

    return molesoutput



#helper function 3: find limiting reactant

def findlimreac(reaction: Reaction) -> Compound:

    ‘’’

    Purpose: Takes in a reaction dataclass and compares the amount of each reactant in moles (using functions mass to mole and mole to mole) to find which one produces more of product 1.

    Args: reaction (Reaction) - A Reaction dataclass which contains 4 compounds reac1/2and prod1/2

    Returns: reaction.reac1.name (str) - the name of the first reactant

reaction.reac2.name (str) - the name of the second reactant

    ‘’’

    if moletomole(masstomole(reaction.reac1.mass, reac1.molarmass), reaction.reac1.coeff, reaction.prod1.coeff)) >= moletomole(masstomole(reaction.reac2.mass, reaction.reac2.molarmass), reaction.reac2.coeff, reaction. prod1.coeff))
```

```
        return reaction.reac1

    elif moletomole(masstomole(reaction.reac2.mass, reac2.molarmass), reaction.reac2.coeff,
reaction.prod1.coeff)) > moletomole(masstomole(reaction.reac1.mass,
reaction.reac1.molarmass), reaction.reac1.coeff, reaction. prod1.coeff))

        return reaction.reac2



#helper function 4: find theoretical yield

def findtheoyield(reaction: Reaction) -> float:

‘’’

moles limreac/1(moles prod1/moles limreac)(g prod1/moles prod1)

‘’’

    limreac = findlimreac(reaction)

    product = reaction.prod1

    theoyield = masstomole(limreac.mass) * product.coeff / limreac.coeff * product.mass /
product.coeff

    return theoyield



#helper function 5: find percent yield

def findpercentyield(reaction: Reaction) -> float:

‘’’

abc

‘’’

theoyield = findtheoyield(reaction)

percentyield = theoyield / reaction.prod1.mass * 100

return percentyield
```

how does this look? helper functions section of my limiting reactant coding project
…

Overall, your helper functions are structured well! You have separated the five major calculations into individual functions, which is exactly what you want for a chemistry stoichiometry calculator.
Your overall approach is correct, but there are a few important chemistry and Python errors that need to be fixed before the functions will work.
I'll go through them individually while keeping your original structure and variable names.

1. masstomole

Chemistry and logic correct

```
def masstomole(mass: float, molarmass: float) -> float:
  amountinmoles = mass / molarmass
  return amountinmoles
```

This is completely correct.
Moles=Mass (g)Molar mass (g/mol)
Moles=
Molar mass (g/mol)
Mass (g)

No changes needed.

2. moletomole

Chemistry and logic correct

```
def moletomole(molesinput: float, coeffinput: int, coeffoutput: int) -> float:
  molesoutput = molesinput * coeffoutput / coeffinput
  return molesoutput
```

Also correct!
You're using the coefficients from the balanced chemical equation to convert between moles of reactant and moles of product.
Moles output=Moles input×Coefficient outputCoefficient input
Moles output=Moles input×
Coefficient input
Coefficient output

No changes needed.

3. findlimreac

Needs corrections
There are several issues here.
First, your Python errors:

- reac1.molarmass should be reaction.reac1.molarmass.
- reac2.molarmass should be reaction.reac2.molarmass.
- You have some extra closing parentheses.
- You're missing colons after your if and elif conditions.

Second, there's a chemistry error.
You currently have:

```
if amount1 >= amount2:
  return reaction.reac1
```

But remember:

- The limiting reactant is the reactant that produces the smaller amount of product.
- The excess reactant produces the larger amount of product.

Therefore, you want to return the reactant that produces LESS product.
Here's how I'd write your function:

```
def findlimreac(reaction: Reaction) -> Compound:
  amount1 = moletomole(masstomole(reaction.reac1.mass, reaction.reac1.molarmass), reaction.reac1.coeff, reaction.prod1.coeff)

  amount2 = moletomole(masstomole(reaction.reac2.mass, reaction.reac2.molarmass), reaction.reac2.coeff, reaction.prod1.coeff)

  if amount1 <= amount2:
     return reaction.reac1
  else:
     return reaction.reac2
```

Notice that I created two variables, amount1 and amount2.
This makes your code easier to read and avoids repeating the same calculations.
Also, your return type of Compound is correct because you're returning the entire Compound object, not just its name.

4. findtheoyield

Needs chemistry corrections
Your intended calculation is correct in concept, but there are a couple of problems with your formula.
You wrote:

```
theoyield = masstomole(limreac.mass) * product.coeff / limreac.coeff * product.mass / product.coeff
```

There are three problems:

1. masstomole requires TWO arguments: mass and molar mass.
2. You need to use the product's molar mass, not its actual mass.
3. You should not cancel out the product coefficient by multiplying and dividing by it.

Your calculation should follow these three steps:
Step 1: Convert limiting reactant mass into moles.
Mass limiting reactantMolar mass limiting reactant
Molar mass limiting reactant
Mass limiting reactant

Step 2: Convert moles of limiting reactant into moles of product.
×Coefficient productCoefficient reactant
×
Coefficient reactant
Coefficient product

Step 3: Convert moles of product into grams.
×Molar mass product
×Molar mass product
Your corrected function:

```
def findtheoyield(reaction: Reaction) -> float:
  limreac = findlimreac(reaction)
  product = reaction.prod1
```

theoyield = masstomole(limreac.mass, limreac.molarmass) * product.coeff / limreac.coeff * product.molarmass

return theoyield

This gives you the theoretical yield in grams.

5. findpercentyield

Formula reversed

You wrote:

percentyield = theoyield / reaction.prod1.mass * 100

You have the correct components, but they're reversed.

The percent yield equation is:

Percent yield=Actual yieldTheoretical yield×100

Percent yield=

Theoretical yield

Actual yield

×100

Your reaction.prod1.mass represents the actual yield, while theoyield represents the theoretical yield.

Corrected function:

```
def findpercentyield(reaction: Reaction) -> float:
  theoyield = findtheoyield(reaction)

  percentyield = reaction.prod1.mass / theoyield * 100

  return percentyield
```

6. Your dataclasses

One small Python issue: your first dataclass is missing a colon.

You currently have:

```
@dataclass
class Compound
```

It should be:

```
@dataclass
class Compound:
```

Your Reaction dataclass looks correct.

7. Overall structure

Here's how your five helper functions fit together:

findpercentyield(reaction)

Calculates the final percent yield

findtheoyield(reaction)

Calculates theoretical yield in grams

findlimreac(reaction)

Identifies the limiting reactant

masstomole()

Grams → Moles
moletomole()
Moles → Moles

8. A few additional things to consider

Before moving on to your Drafter routes, I'd recommend considering these details:
1. What happens if both reactants produce exactly the same amount of product? Your current <= condition will return the first reactant. That's reasonable, although technically neither reactant is limiting when they are present in exactly stoichiometric proportions.
2. What happens if the user enters a mass of zero? Your functions can handle zero mass, but percent yield will involve division by zero if theoretical yield is zero. You may want to prevent that input.
3. What about your second product? Your current functions calculate theoretical yield using only prod1. That's perfectly fine if your website is designed to calculate the yield of just one selected product.
My overall assessment: Your five-function structure is good, and your first two functions are already correct. The biggest thing to fix was the limiting reactant comparison and the theoretical yield calculation. Your project is coming together logically!

```
from bakery import assert_equal

from dataclasses import dataclass

@dataclass

class Compound:

    name: str

    mass: float

    molarmass: float

    coeff: int

@dataclass

class Reaction:

    reac1: Compound

    reac2: Compound

    prod1: Compound

    prod2: Compound

#helper function 1: mass to mole

def masstomole(mass: float, molarmass: float) -> float:
```

```
    ‘’’

    Purpose: Takes in a mass of element/compound/etc, and calculates the amount it equals in moles of the same element/compound using the molar mass.

Stoich: mass (g)/1(mol (mol)/molarmass (g))=mol

    Args: mass (float) - the amount of stuff there is in grams

molarmass (float) - the amount of grams there are in 1 mol of the compound

    Returns: amountinmoles (float) - the amount of stuff there is in moles

    ‘’’

    amountinmoles = mass / molarmass

    return amountinmoles



#helper function 2: mole to mole

def moletomole(molesinput: float, coeffinput: int, coeffoutput: int) -> float:

    ‘’’

    Purpose: Takes in the amount of input material in moles, as well as the coefficients of the input and output materials. Completes a calculation to give the amount of output material in moles.

Stoich: molesinput (mol input)/1(coeffoutput (mol output)/coeffinput (mol input)) = mol output

    Args: molesinput (float) - the amount of the input material in moles

coeffinput (int) - the coefficient of the input material from the balanced reaction

coeffoutput (int) - the coefficient of the output material from the balanced reaction

    Returns: molesoutput (float) -

    ‘’’

    molesoutput = molesinput * coeffoutput / coeffinput

    return molesoutput



#helper function 3: find limiting reactant
```

```
def findlimreac(reaction: Reaction) -> Compound:
    ‘’’
    Purpose: Takes in a reaction dataclass and compares the amount of each reactant in moles (using functions mass to mole and mole to mole) to find which one produces more of product 1.
    Args: reaction (Reaction) - A Reaction dataclass which contains 4 compounds reac1/2and prod1/2
    Returns: reaction.reac1.name (str) - the name of the first reactant
reaction.reac2.name (str) - the name of the second reactant
    ‘’’
    amount1 = moletomole(masstomole(reaction.reac1.mass, reaction.reac1.molarmass), reaction.reac1.coeff, reaction.prod1.coeff)
    amount2 = moletomole(masstomole(reaction.reac2.mass, reaction.reac2.molarmass), reaction.reac2.coeff, reaction.prod1.coeff)
if amount1 >= amount2:
        return reaction.reac2
    elif amount2 > amount1:
        return reaction.reac1

#helper function 4: find theoretical yield
def findtheoyield(reaction: Reaction) -> float:
‘’’
moles limreac/1(moles prod1/moles limreac)(g prod1/moles prod1)
‘’’
    limreac = findlimreac(reaction)
    product = reaction.prod1
    theoyield = masstomole(limreac.mass, limreac.molarmass) * product.coeff / limreac.coeff * product.molarmass
    return theoyield
```

```
#helper function 5: find percent yield

def findpercentyield(reaction: Reaction) -> float:

‘’’

abc

‘’’

theoyield = findtheoyield(reaction)

percentyield = reaction.prod1.mass / theoyield * 100

return percentyield
```

…

This is a significant improvement! Your chemistry calculations are now correct, and your functions are working together logically. There are just a few remaining things I'd fix before moving on to testing.

I'll focus on actual errors rather than rewriting code that is already correct.

1. masstomole and moletomole

Correct

Both functions are good. No changes needed to the calculations or arguments.

2. findlimreac

One important issue

Your comparison is now correct:

```
if amount1 >= amount2:
  return reaction.reac2
elif amount2 > amount1:
  return reaction.reac1
```

This correctly identifies the reactant that produces the smaller amount of product.

However, your indentation is incorrect in the code you pasted.

Both if and elif need to be indented inside the function.

Also, your purpose statement says you find which reactant produces MORE product. Change that to LESS product because you're finding the limiting reactant.

Your corrected function:

```
def findlimreac(reaction: Reaction) -> Compound:
  amount1 = moletomole(masstomole(reaction.reac1.mass, reaction.reac1.molarmass),
reaction.reac1.coeff, reaction.prod1.coeff)

  amount2 = moletomole(masstomole(reaction.reac2.mass, reaction.reac2.molarmass),
reaction.reac2.coeff, reaction.prod1.coeff)

  if amount1 >= amount2:
```

```
    return reaction.reac2
  elif amount2 > amount1:
    return reaction.reac1
```
One optional improvement: You could replace elif with else, since if the first condition is false, the second condition must be true.

3. findtheoyield

Correct
Your calculation is now correct!
```
def findtheoyield(reaction: Reaction) -> float:
  limreac = findlimreac(reaction)
  product = reaction.prod1

  theoyield = masstomole(limreac.mass, limreac.molarmass) * product.coeff / limreac.coeff *
product.molarmass

  return theoyield
```
You're correctly converting:

1. Grams of limiting reactant → moles of limiting reactant.
2. Moles of limiting reactant → moles of product.
3. Moles of product → grams of product.

No changes needed.

4. findpercentyield

Correct
Your formula is now correct:
```
def findpercentyield(reaction: Reaction) -> float:
  theoyield = findtheoyield(reaction)
  percentyield = reaction.prod1.mass / theoyield * 100
  return percentyield
```
Just remember that reaction.prod1.mass represents the actual mass of product obtained experimentally.

5. Remaining things to fix

Here's my checklist:

- masstomole
- moletomole
- Limiting reactant comparison
- Theoretical yield calculation
- Percent yield calculation
- Fix indentation in findlimreac.
- Make sure all your docstrings use three regular quotation marks (""") rather than curly quotation marks (‘’‘).
- Indent the body of findtheoyield and findpercentyield properly.
- Update the purpose statement for findlimreac.

One additional chemistry consideration: If both reactants produce exactly the same amount of product, neither is technically limiting. Your current function returns reaction.reac2 in that situation. You may eventually want to account for this.

6. What I'd do next: test your helper functions

Before building your Drafter routes, I'd test these functions with a known chemical reaction.
For example:
2H2+O2→2H2O
2H
2

+O
2

→2H
2

O
Suppose you have:

- 4.00 g of
- H2
- H
- 2
-
- 16.0 g of
- O2
- O
- 2
-
- 18.0 g of actual
- H2O
- H
- 2
-
- O produced

You can use this to test all five functions.
The expected results are:

Calculation	Expected result
Moles of H_2	1.984 mol
Moles of O_2	0.500 mol
Limiting reactant	O_2
Theoretical yield H_2O	18.0 g
Percent yield	100%

This is a particularly useful test because it checks whether your limiting reactant function correctly identifies the second reactant.

My recommendation: Your helper function section is essentially complete after the indentation and docstring fixes. I'd move on to writing assert_equal tests before starting your website routes.

```
from bakery import assert_equal

from dataclasses import dataclass

@dataclass

class Compound:

    name: str

    mass: float

    molarmass: float

    coeff: int

@dataclass

class Reaction:

    reac1: Compound

    reac2: Compound

    prod1: Compound

    prod2: Compound

#helper function 1: mass to mole

def masstomole(mass: float, molarmass: float) -> float:

    ‘’’

    Purpose: Takes in a mass of element/compound/etc, and calculates the amount it equals in moles of the same element/compound using the molar mass.

Stoich: mass (g)/1(mol (mol)/molarmass (g))=mol

    Args: mass (float) - the amount of stuff there is in grams

molarmass (float) - the amount of grams there are in 1 mol of the compound

    Returns: amountinmoles (float) - the amount of stuff there is in moles

    ‘’’
```

```
    amountinmoles = mass / molarmass

    return amountinmoles


#helper function 2: mole to mole

def moletomole(molesinput: float, coeffinput: int, coeffoutput: int) -> float:

    ‘’’

    Purpose: Takes in the amount of input material in moles, as well as the coefficients of the input and output materials. Completes a calculation to give the amount of output material in moles.

Stoich: molesinput (mol input)/1(coeffoutput (mol output)/coeffinput (mol input)) = mol output

    Args: molesinput (float) - the amount of the input material in moles

coeffinput (int) - the coefficient of the input material from the balanced reaction

coeffoutput (int) - the coefficient of the output material from the balanced reaction

    Returns: molesoutput (float) -

    ‘’’

    molesoutput = molesinput * coeffoutput / coeffinput

    return molesoutput


#helper function 3: find limiting reactant

def findlimreac(reaction: Reaction) -> Compound:

    ‘’’

    Purpose: Takes in a reaction dataclass and compares the amount of each reactant in moles (using functions mass to mole and mole to mole) to find which one produces more of product 1.

    Args: reaction (Reaction) - A Reaction dataclass which contains 4 compounds reac1/2and prod1/2

    Returns: reaction.reac1.name (str) - the name of the first reactant

reaction.reac2.name (str) - the name of the second reactant

    ‘’’
```

```
    amount1 = moletomole(masstomole(reaction.reac1.mass, reaction.reac1.molarmass),
reaction.reac1.coeff, reaction.prod1.coeff)

    amount2 = moletomole(masstomole(reaction.reac2.mass, reaction.reac2.molarmass),
reaction.reac2.coeff, reaction.prod1.coeff)

    if amount1 >= amount2:

        return reaction.reac2

    elif amount2 > amount1:

        return reaction.reac1



#helper function 4: find theoretical yield

def findtheoyield(reaction: Reaction) -> float:

‘’’

moles limreac/1(moles prod1/moles limreac)(g prod1/moles prod1)

‘’’

    limreac = findlimreac(reaction)

    product = reaction.prod1

    theoyield = masstomole(limreac.mass, limreac.molarmass) * product.coeff / limreac.coeff *
product.molarmass

    return theoyield



#helper function 5: find percent yield

def findpercentyield(reaction: Reaction) -> float:

‘’’

abc

‘’’

    theoyield = findtheoyield(reaction)

    percentyield = reaction.prod1.mass / theoyield * 100
```

```
    return percentyield
```
…
Your findlimreac indentation is fixed now! The five calculations are also logically correct. There are just three remaining issues I'd address.

1. Fix the indentation in findtheoyield

Your docstring and function body need to be indented inside the function.

```
def findtheoyield(reaction: Reaction) -> float:
  """
  moles limreac/1(moles prod1/moles limreac)(g prod1/moles prod1)
  """
  limreac = findlimreac(reaction)
  product = reaction.prod1
  theoyield = masstomole(limreac.mass, limreac.molarmass) * product.coeff / limreac.coeff *
product.molarmass
  return theoyield
```

2. Fix your docstrings

Throughout your code, you're using curly quotation marks:
“”
Python requires three straight quotation marks:
"""
Make this change in all five functions.

3. Correct your purpose statement

Your findlimreac purpose statement still says:
find which one produces more of product 1.
Change more to less because the limiting reactant produces the smaller amount of product.

Final assessment

Helper function 1: masstomole
Correct
Helper function 2: moletomole
Correct
Helper function 3: findlimreac
Logic correct
Helper function 4: findtheoyield
Logic correct
Helper function 5: findpercentyield
Correct
One thing I would add before moving on: Write assert_equal tests for each function. In particular, test findlimreac with examples where the first reactant is limiting and where the second reactant is limiting.
Your helper functions are very close to finished! I would avoid making further structural changes until you've tested them.

```
from bakery import assert_equal
```

```
from dataclasses import dataclass

@dataclass

class Compound:

    name: str

    mass: float

    molarmass: float

    coeff: int

@dataclass

class Reaction:

    reac1: Compound

    reac2: Compound

    prod1: Compound

    prod2: Compound

reaction1 = Reaction(Compound(“Al”, 124.0, 27.0, 2), Compound(“Fe2O3”, 601.0, 104.0, 1), Compound(“Al2O3”, 234.2, 102.0, 1), Compound(“Fe”, 1.0, 1.0, 2))

#helper function 1: mass to mole

def masstomole(mass: float, molarmass: float) -> float:

    “””

    Purpose: Takes in a mass of element/compound/etc, and calculates the amount it equals in moles of the same element/compound using the molar mass.

Stoich: mass (g)/1(mol (mol)/molarmass (g))=mol

    Args: mass (float) - the amount of stuff there is in grams

molarmass (float) - the amount of grams there are in 1 mol of the compound

    Returns: amountinmoles (float) - the amount of stuff there is in moles

    “””

    amountinmoles = mass / molarmass
```

```
    return amountinmoles

assert_equal(masstomole(10.0, 20.0), 0.5)

assert_equal(masstomole(20.0, 10.0), 2.0)



#helper function 2: mole to mole

def moletomole(molesinput: float, coeffinput: int, coeffoutput: int) -> float:

    “””

    Purpose: Takes in the amount of input material in moles, as well as the coefficients of the input and output materials. Completes a calculation to give the amount of output material in moles.

Stoich: molesinput (mol input)/1(coeffoutput (mol output)/coeffinput (mol input)) = mol output

    Args: molesinput (float) - the amount of the input material in moles

coeffinput (int) - the coefficient of the input material from the balanced reaction

coeffoutput (int) - the coefficient of the output material from the balanced reaction

    Returns: molesoutput (float) -

    “””

    molesoutput = molesinput * coeffoutput / coeffinput

    return molesoutput

assert_equal(moletomole(10.0, 2, 4), 20.0)

assert_equal(moletomole(10.0, 4, 2), 5.0)



#helper function 3: find limiting reactant

def findlimreac(reaction: Reaction) -> Compound:

    “””

    Purpose: Takes in a reaction dataclass and compares the amount of each reactant in moles (using functions mass to mole and mole to mole) to find which one produces less of product 1.

    Args: reaction (Reaction) - A Reaction dataclass which contains 4 compounds reac1/2and prod1/2
```

```
    Returns: reaction.reac1.name (str) - the name of the first reactant

reaction.reac2.name (str) - the name of the second reactant

    “””

    amount1 = moletomole(masstomole(reaction.reac1.mass, reaction.reac1.molarmass),
reaction.reac1.coeff, reaction.prod1.coeff)

    amount2 = moletomole(masstomole(reaction.reac2.mass, reaction.reac2.molarmass),
reaction.reac2.coeff, reaction.prod1.coeff)

    if amount1 >= amount2:

        return reaction.reac2

    elif amount2 > amount1:

        return reaction.reac1

assert_equal(findlimreac(reaction1), Compound(“Al”, 124.0, 27.0, 2))



#helper function 4: find theoretical yield

def findtheoyield(reaction: Reaction) -> float:

    “””

    moles limreac/1(moles prod1/moles limreac)(g prod1/moles prod1)

    “””

    limreac = findlimreac(reaction)

    product = reaction.prod1

    theoyield = masstomole(limreac.mass, limreac.molarmass) * product.coeff / limreac.coeff *
product.molarmass

    return theoyield

assert_equal(findtheoyield(reaction1), 234.2)



#helper function 5: find percent yield

def findpercentyield(reaction: Reaction) -> float:
```

```
    “””
    finds the percent yield of a given reaction
    “””
    theoyield = findtheoyield(reaction)
    percentyield = reaction.prod1.mass / theoyield * 100
    return percentyield

assert_equal(findpercentyield(reaction1), 100.0)
```

…

You're making good progress with the testing! You've now added assert_equal tests for all five helper functions, and your overall testing structure is good.

However, I caught two important problems with reaction1 and its expected results. Your first two helper functions have good tests, but the last three tests need some attention.

Let's work through them.

1. masstomole — Correct

Both tests are mathematically correct:

```
assert_equal(masstomole(10.0, 20.0), 0.5)
assert_equal(masstomole(20.0, 10.0), 2.0)
```

2. moletomole — Correct

Both tests are also correct:

```
assert_equal(moletomole(10.0, 2, 4), 20.0)
assert_equal(moletomole(10.0, 4, 2), 5.0)
```

These test both increasing and decreasing the mole amount based on the coefficients.

3. findlimreac — Logic correct, test correct

Your reaction is:

```
2Al+Fe2O3→Al2O3+2Fe
2Al+Fe
2

O
3

→Al
2

O
3

+2Fe
```

Your function correctly identifies aluminum as the limiting reactant.
However, there is a problem with your Fe2O3 molar mass.
You entered:
Compound("Fe2O3", 601.0, 104.0, 1)
The molar mass should be approximately 159.7 g/mol, not 104.0 g/mol.
Corrected:
Compound("Fe2O3", 601.0, 159.7, 1)
Your limiting reactant test should still pass because aluminum is still limiting.
I'd also recommend adding another test where Fe_2O_3 is the limiting reactant. That way you test both branches of your if statement.

4. findtheoyield — Small testing issue

Your function's calculation is correct.
However, let's calculate the theoretical yield using your aluminum values:
124.027.0×12×102.0
27.0
124.0

×
2
1

×102.0
=234.2222 g
=234.2222 g
You currently have:
assert_equal(findtheoyield(reaction1), 234.2)
Depending on how assert_equal handles floating-point numbers, this might fail because the actual result is approximately 234.2222, not exactly 234.2.
I'd recommend:
assert_equal(round(findtheoyield(reaction1), 1), 234.2)
This tests the answer to one decimal place.

5. findpercentyield — Expected answer needs correction

This is the biggest issue with your tests.
Your actual product mass is:
Compound("Al2O3", 234.2, 102.0, 1)
Your theoretical yield is approximately 234.2222 g.
Therefore:
234.2234.2222×100
234.2222
234.2

×100
=99.9905%
=99.9905%
Your current test:
assert_equal(findpercentyield(reaction1), 100.0)
would not be correct.

Instead, use:
assert_equal(round(findpercentyield(reaction1), 2), 99.99)

6. One additional correction

Your second product currently has:
Compound("Fe", 1.0, 1.0, 2)
Iron's molar mass is approximately 55.85 g/mol.
I'd change it to:
Compound("Fe", 1.0, 55.85, 2)
Even though your current helper functions don't use the second product, it's important to keep your reaction data accurate.

Final assessment

Your testing progress
masstomole
Passed
moletomole
Passed
findlimreac
Passed (with corrected molar mass)
findtheoyield
Adjust expected precision
findpercentyield
Adjust expected percentage
One last Python reminder: Make sure your quotation marks are straight quotation marks ("), not curly quotation marks (“ ”). Python won't recognize curly quotation marks as string delimiters.

What I'd recommend next

Once you've corrected these tests, I'd consider your five helper functions finished.
The next step should be testing your functions with a second reaction, particularly one where the second reactant is limiting.
That will give you much more confidence that your website can handle different chemical reactions rather than just your original example.

```
@dataclass

class State:

    message: str

    balancedreaction: Reaction

    limreac: Compound

    theoyield: float

    percentyield: float
```

```
#route function 1: index

@route

def index(state: State) -> Page:

    """

    This function produces the initial home page of the website, which has 5 buttons on it.

The input values button takes the user to the input values page, where they can input the values for one compound of the reaction.

The calculate answers button takes the user to the calculate answers page, where they can calculate the answers for the reaction that they input.

The view answers button takes the user to the view answers page where the identified limiting reactant, theoretical yield, and percent yield are shown in a text box.

The process explanation button takes the user to the process explanation page where there is a text box explaining how the answers were calculated

    """

return Page(state)



#route function 2: reset

@route

def reset(state: State) -> Page:

    return Page(state)



#route function 3: home

@route

def home(state: State) -> Page:

    return Page(state)



#route function 4: compoundinput
```

```
@route

def compoundinput(state: State) -> Page:

    """

    Upon clicking the input values button on the home page, the user is taken to the compound input page, where there are 4 buttons, one to input the name of the compound, one to input the mass, one to input the molar mass, and one to input the coefficient in the balanced reaction. After going through each button, the user can press the 5th home button to return to the home page and see an updated balanced reaction with one compound filled in.

    """

    return Page(state)



#route function 5: calculate answers

@route

def calculateanswers(state: State) -> Page:

    """

    Upon clicking the calculate answers button on the home page, the user is taken to the calculate answers page, where there are 2 buttons. 1 will calculate the answers, which this function does, and the other returns the user to the home page.

    """

    return Page(state)



#route function 6: view answers

@route

def viewanswers(state: State) -> Page:

    """

    Upon clicking the view answers button on the home page, the user is taken to the calculate answers page, where there is a textbox displaying the answers for the specified limiting reactant, theoretical yield, and percent yield. There is also a button that returns to the home page

    """
```

```
    return Page(state)


#route function 7: explain process

@route

def explainprocess(state: State) -> Page:

    """

    Upon clicking the process explanation button on the home page, the user is taken to the process explanation page, where there is a textbox that shows how the calculations were completed, and a button for returning home.

    """

    return Page(state)



start_server(State(message = "Welcome to Home Page, choose what to do from options below")
```

…

You're now moving from the helper-function portion of your project into the Drafter website portion. Your seven route functions give you a good starting structure, and your page ideas are consistent with your project's purpose.

There are a few important things to understand before you start adding buttons and input fields. The biggest issue right now is that your route functions don't actually navigate anywhere yet. They currently return Page(state), which just displays a page without defining its contents or providing navigation.

Let's go through your design.

1. Your State dataclass

You currently have:

```
@dataclass
class State:
  message: str
  balancedreaction: Reaction
  limreac: Compound
  theoyield: float
  percentyield: float
```

This is a reasonable starting point, but there is one important consideration.

Your balancedreaction contains four Compound objects, so you already have somewhere to store:

- Reactant 1
- Reactant 2
- Product 1
- Product 2

You don't need separate State variables for all four compounds.
However, your limreac, theoyield, and percentyield values need initial values when you create your State.
For example, you could eventually initialize them with:

```
State(
 @dataclass
class State:
   message: str
   balancedreaction: Reaction
   limreac: Compound
   theoyield: float
   percentyield: float

#default state
state0 = State("Welcome to Home Page of Limiting Reactant Calculator, choose your action below", Reaction(Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), None, 0.0, 0.0)



#route function 1: index
@route
def index(state: State) -> Page:
   """
   This function produces the initial home page of the website, which has 5 buttons on it.

The input values button takes the user to the input values page, where they can input the values for one compound of the reaction.

The calculate answers button takes the user to the calculate answers page, where they can calculate the answers for the reaction that they input.

The view answers button takes the user to the view answers page where the identified limiting reactant, theoretical yield, and percent yield are shown in a text box.

The process explanation button takes the user to the process explanation page where there is a text box explaining how the answers were calculated
   """
return Page(state0, [Text(state.message), Button("Input Values", compoundinput), Button("Calculate Answers", calculateanswers), Button("View Answers", viewanswers), Button("Process Explanation", explainprocess), Button("Reset Values", reset)]



#route function 2: reset
@route
def reset(state: State) -> Page:
   return Page(state0)
```

This assumes reaction1 is your previously defined example.

2. Your seven route functions

Here's my assessment of each route.

Route	Purpose	Assessment
index	Main home page	Good
reset	Reset website values	Needs functionality
home	Return to home page	Potentially redundant
compoundinput	Enter compound information	Good, needs input handling
calculateanswers	Calculate results	Good, needs calculations
viewanswers	Display results	Good
explainprocess	Explain calculations	Good

Route 1: index

Your index function should display the five buttons you described.
Eventually, it will look something like this:

```
@route
def index(state: State) -> Page:
  return Page(state, [
    Header("Limiting Reactant Calculator"),
    Text(state.message),
    Button("Input Values", compoundinput),
    Button("Calculate Answers", calculateanswers),
    Button("View Answers", viewanswers),
    Button("Explain Process", explainprocess),
    Button("Reset", reset)
  ])
```

This is an example of how Drafter buttons connect to other routes.
Notice that Button takes the name of another route function. This is what makes navigation work.
You'll need the appropriate Drafter components imported.

Route 2: reset

Currently:

```
@route
def reset(state: State) -> Page:
  return Page(state)
```

This doesn't actually reset anything.
Eventually, this function should restore your original/default values.
For example, it could reset the balanced reaction and calculated results to their initial values.

Route 3: home

Currently, this is identical to your reset function.
These two routes should have different purposes:

- home: Returns the user to the main page without deleting their information.
- reset: Clears their information and restores the initial state.

You could also make home unnecessary by having all your other pages contain a button that navigates directly to index.

Route 4: compoundinput

This is probably going to be the most complicated route.
Your description says the user needs to enter:

1. Compound name
2. Mass
3. Molar mass
4. Coefficient

However, remember that your reaction contains FOUR compounds.
That means you need to collect information for:

- Reactant 1
- Reactant 2
- Product 1
- Product 2

You'll need a way to distinguish which compound the user is currently entering.
One possibility is to have four separate input pages, or have one input page with four clearly labeled sections.
I would recommend the second option because it keeps your website simpler.

Route 5: calculateanswers

This function should call your existing helper functions!
For example, its calculation section will eventually contain:
state.limreac = findlimreac(state.balancedreaction)

state.theoyield = findtheoyield(state.balancedreaction)

state.percentyield = findpercentyield(state.balancedreaction)
This is one of the biggest advantages of how you've structured your project: you've already written the chemistry calculations, so you don't need to repeat them in your routes.

Route 6: viewanswers

Your description is good.
This page should display:

- Limiting reactant
- Theoretical yield (grams)
- Percent yield

For example, you could use Drafter's Text component to display the results.
Text("Limiting Reactant: " + state.limreac.name)
Text("Theoretical Yield: " + str(state.theoyield) + " g")
Text("Percent Yield: " + str(state.percentyield) + "%")

Route 7: explainprocess

I like this feature because it makes your website more educational rather than simply giving users answers.
You could display:

1. Conversion of reactant masses into moles.
2. Comparison of the possible product amounts.
3. Identification of the limiting reactant.
4. Calculation of theoretical yield.
5. Calculation of percent yield.

You can use your existing helper functions to determine the numerical values displayed in the explanation.

3. Important: Your start_server statement

At the bottom, you currently have:
start_server(State(message = "Welcome to Home Page, choose what to do from options below")
There are two problems:

1. You're missing the other four required State arguments.
2. You're missing a closing parenthesis.

Assuming you want to start with your previously defined reaction1, you could initialize it like this:

```
start_server(State(
  message="Welcome to Home Page, choose what to do from options below",
  balancedreaction=reaction1,
  limreac=reaction1.reac1,
  theoyield=0.0,
  percentyield=0.0
))
```

This gives every State field an initial value.

4. My overall recommendations

Before you start implementing the routes, here's what I'd prioritize:

Priority	Task
1	Finish the State initialization
2	Implement the five home-page buttons
3	Implement compound input
4	Connect your helper functions to calculateanswers
5	Display results in viewanswers
6	Implement explainprocess
7	Make reset restore default values

One important distinction: Having seven @route functions satisfies a requirement for seven routes, assuming your assignment counts each of these functions. However, simply defining seven routes doesn't mean all seven are functional yet.
Your next major challenge is connecting the buttons and input fields to your existing State dataclass.
I would start with index and compoundinput, since the rest of your website depends on receiving the chemical information correctly.

```
@dataclass

class State:

    message: str

    balancedreaction: Reaction

    limreac: Compound

    theoyield: float

    percentyield: float

#default state

state0 = State("Welcome to Home Page of Limiting Reactant Calculator, choose your action below", Reaction(Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), None, 0.0, 0.0)



#route function 1: index

@route

def index(state: State) -> Page:

    """

    This function produces the initial home page of the website, which has 5 buttons on it.
```

The input values button takes the user to the input values page, where they can input the values for one compound of the reaction.

The calculate answers button takes the user to the calculate answers page, where they can calculate the answers for the reaction that they input.

The view answers button takes the user to the view answers page where the identified limiting reactant, theoretical yield, and percent yield are shown in a text box.

The process explanation button takes the user to the process explanation page where there is a text box explaining how the answers were calculated

```
    """
return Page(state0, [Text(state.message), Button("Input Values", compoundinput),
Button("Calculate Answers", calculateanswers), Button("View Answers", viewanswers),
Button("Process Explanation", explainprocess), Button("Reset Values", reset)]


#route function 2: reset
@route
def reset(state: State) -> Page:
    return Page(state0)


#route function 3: home
@route
def home(state: State) -> Page:
    return Page(state)


#route function 4: compoundinput
@route
def compoundinput(state: State) -> Page:
    """
    Upon clicking the input values button on the home page, the user is taken to the compound
input page, where there are 4 buttons, one to input the name of the compound, one to input the
mass, one to input the molar mass, and one to input the coefficient in the balanced reaction.
After going through each button, the user can press the 5th home button to return to the home
page and see an updated balanced reaction with one compound filled in.
    """
    return Page(state)


#route function 5: calculate answers
```

```
@route

def calculateanswers(state: State) -> Page:

    """

    Upon clicking the calculate answers button on the home page, the user is taken to the calculate answers page, where there are 2 buttons. 1 will calculate the answers, which this function does, and the other returns the user to the home page.

    """

    return Page(state)



#route function 6: view answers

@route

def viewanswers(state: State) -> Page:

    """

    Upon clicking the view answers button on the home page, the user is taken to the calculate answers page, where there is a textbox displaying the answers for the specified limiting reactant, theoretical yield, and percent yield. There is also a button that returns to the home page

    """

    return Page(state)



#route function 7: explain process

@route

def explainprocess(state: State) -> Page:

    """

    Upon clicking the process explanation button on the home page, the user is taken to the process explanation page, where there is a textbox that shows how the calculations were completed, and a button for returning home.

    """

    return Page(state)
```

start_server(State(message = "Welcome to Home Page, choose what to do from options below")
…
You're getting closer! I like that you've introduced state0 as a default state. This is a good way to handle resetting your website rather than manually resetting every variable.
However, there are three major errors in your updated code, plus a couple of smaller improvements.

1. Your state0 initialization has too many arguments

You currently have:
state0 = State("Welcome...", Reaction(...), None, 0.0, 0.0)
But your Reaction dataclass only has four fields:

```
@dataclass
class Reaction:
  reac1: Compound
  reac2: Compound
  prod1: Compound
  prod2: Compound
```

Your current initialization appears to put None, 0.0, and 0.0 inside the Reaction constructor. Those values belong to State, not Reaction.
Here's the corrected initialization:

```
state0 = State(
  "Welcome to Home Page of Limiting Reactant Calculator, choose your action below",
  Reaction(
     Compound("", 0.0, 0.0, 0),
     Compound("", 0.0, 0.0, 0),
     Compound("", 0.0, 0.0, 0),
     Compound("", 0.0, 0.0, 0)
  ),
  Compound("", 0.0, 0.0, 0),
  0.0,
  0.0
)
```

Now you have exactly five arguments for State.

2. Your index function

Your buttons are correctly connected to your other routes!
However, there are two things to fix.
First: You are missing indentation before return Page.
Second: You should return Page(state, ...) instead of Page(state0, ...).
Why?

- state represents the current information in your website.
- state0 represents the original default information.

If you use state0 every time someone visits the home page, you could accidentally erase their entered information.
Corrected:

```
@route
```

```
def index(state: State) -> Page:
  return Page(state, [
      Text(state.message),
      Button("Input Values", compoundinput),
      Button("Calculate Answers", calculateanswers),
      Button("View Answers", viewanswers),
      Button("Process Explanation", explainprocess),
      Button("Reset Values", reset)
  ])
```

3. Your reset function

This is actually one of the functions you've implemented correctly!

```
@route
def reset(state: State) -> Page:
  return Page(state0)
```

This returns the user to the default state.
No changes needed.

4. Your home function

Also correct:

```
@route
def home(state: State) -> Page:
  return Page(state)
```

This preserves the current state.
However, you could simply have your other pages use:
Button("Return Home", index)
That would make your separate home route unnecessary, unless your assignment specifically requires seven routes.

5. Your start_server

At the bottom, you currently have two different State initializations mixed together.
You only need:
start_server(state0)
This is much simpler because you've already defined your default state!

6. Your updated route structure

Here's how your website should work:
Home Page (index)
Five navigation buttons
Input Values
compoundinput
Calculate
calculateanswers
View Answers
viewanswers
Explanation
explainprocess

Reset Values
reset → state0

7. What I recommend doing next

I would leave your State, state0, index, reset, and home functions alone once you've made these corrections.
Your next task should be implementing compoundinput.
One design decision to make first:
How do you want users to enter the four compounds?

- Four separate input sections on one page.
- One compound at a time, with a button to move to the next compound.

I'd lean toward four separate input sections because it will make managing your Reaction dataclass much easier.

```
@dataclass

class State:

    message: str

    balancedreaction: Reaction

    limreac: Compound

    theoyield: float

    percentyield: float

#default state

state0 = State("Welcome to Home Page of Limiting Reactant Calculator, choose your action below", Reaction(Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0)), None, 0.0, 0.0)



#route function 1: index

@route

def index(state: State) -> Page:

    """

    This function produces the initial home page of the website, which has 5 buttons on it.
```

The input values button takes the user to the input values page, where they can input the values for one compound of the reaction.

The calculate answers button takes the user to the calculate answers page, where they can calculate the answers for the reaction that they input.

The view answers button takes the user to the view answers page where the identified limiting reactant, theoretical yield, and percent yield are shown in a text box.

The process explanation button takes the user to the process explanation page where there is a text box explaining how the answers were calculated

"""

return Page(state, [Text(state.message), Button("Input Values", compoundinput), Button("Calculate Answers", calculateanswers), Button("View Answers", viewanswers), Button("Process Explanation", explainprocess), Button("Reset Values", reset)]

#route function 2: reset

@route

def reset(state: State) -> Page:

return Page(state0)

#route function 3: home

@route

def home(state: State) -> Page:

return Page(state)

#route function 4: compoundinput

@route

def compoundinput(state: State) -> Page:

"""

Upon clicking the input values button on the home page, the user is taken to the compound input page, where there are 4 buttons, one to input the name of the compound, one to input the mass, one to input the molar mass, and one to input the coefficient in the balanced reaction. After going through each button, the user can press the 5th home button to return to the home page and see an updated balanced reaction with one compound filled in.

```
    """
    return Page(state, [Text("Go through each input section one time for each compound in the reaction."), Button("Compound Name", input("What is the formula for this compound?"), Button("Compound Mass", input("What is the given, not calculated mass of this compound, in grams?"), Button("Compound Molar Mass", input("What is the molar mass of this compound, in g/mol?"), Button("Compound Coefficient", input("What is the coefficient of this compound in the given balanced reaction?"), Button("Return Home", home)])


#route function 5: calculate answers

@route

def calculateanswers(state: State) -> Page:

    """
    Upon clicking the calculate answers button on the home page, the user is taken to the calculate answers page, where there are 2 buttons. 1 will calculate the answers, which this function does, and the other returns the user to the home page.

    """
    return Page(state, [Text("Press button below to calculate answers and then return home."), Button("Calculate Answers", state.limreac = findlimreac(state.balancedreaction)), Button("Return Home", home)])


#route function 6: view answers

@route

def viewanswers(state: State) -> Page:

    """
    Upon clicking the view answers button on the home page, the user is taken to the calculate answers page, where there is a textbox displaying the answers for the specified limiting reactant, theoretical yield, and percent yield. There is also a button that returns to the home page

    """
    return Page(state, [Text("The limiting reactant is (state.limreac), the theoretical yield is (state.theoyield), and the percent yield is (state.percentyield), Button("Return Home", home)])
```

#route function 7: explain process

@route

def explainprocess(state: State) -> Page:

    """

    Upon clicking the process explanation button on the home page, the user is taken to the process explanation page, where there is a textbox that shows how the calculations were completed, and a button for returning home.

    """

    return Page(state, [Text("The way this process is completed is in 3 major steps. First, you must determine which reactant is limiting, meaning runs out first. This is done by converting the mass of each reactant to moles of reactant and then to moles of product. Whichever reactant produces less moles of product is the limiting reactant. Second, to find the theoretical yield, take the moles of product produced by the limiting reactant and convert to mass of product using molar mass. Third, to find percent yield, use the given mass of product, divide it by the theoretical yield, and multiply by 100.")])

start_server(state0)
…
You're now attempting to make the routes interactive, which is an important step! Your index, reset, and home structure is heading in the right direction, and your process explanation is already written.
However, you've run into an important Drafter concept: buttons and input fields don't work like ordinary Python function calls. They need to connect to route functions that receive and update the State.
I'll go through the issues without changing your overall seven-route design.

1. Default state

Correct
This is now structured correctly:

```
state0 = State(
  "Welcome to Home Page of Limiting Reactant Calculator, choose your action below",
  Reaction(
     Compound("", 0.0, 0.0, 0),
     Compound("", 0.0, 0.0, 0),
     Compound("", 0.0, 0.0, 0),
     Compound("", 0.0, 0.0, 0)
  ),
  None,
  0.0,
  0.0
)
```

You have five State arguments, matching your five dataclass fields.

2. index

Your button arrangement is good, but your return needs to be indented inside the function.

```
@route
def index(state: State) -> Page:
  return Page(state, [
      Text(state.message),
      Button("Input Values", compoundinput),
      Button("Calculate Answers", calculateanswers),
      Button("View Answers", viewanswers),
      Button("Process Explanation", explainprocess),
      Button("Reset Values", reset)
  ])
```

This is the correct general structure for your home page.

3. compoundinput

This is where you have your biggest problem.
You wrote:
Button("Compound Name", input("What is the formula for this compound?"))
The problem is that input() is a regular Python function that waits for input in the console. It is not a Drafter input field.
Drafter uses components such as TextBox and NumberBox to collect information on a webpage.
Also, your Button components should connect to route functions, rather than directly contain input fields.
My recommendation: Use four input fields for each compound, with a button to submit the information.

4. calculateanswers

There is a fundamental Python error in this line:
Button("Calculate Answers", state.limreac = findlimreac(state.balancedreaction))
You cannot put an assignment statement inside the arguments of a Button.
Instead, the route itself should perform the calculations.
Since your home page already has a Calculate Answers button that navigates to this route, you can calculate everything when the user arrives at the page:

```
@route
def calculateanswers(state: State) -> Page:
  state.limreac = findlimreac(state.balancedreaction)
  state.theoyield = findtheoyield(state.balancedreaction)
  state.percentyield = findpercentyield(state.balancedreaction)

  return Page(state, [
      Text("Your calculations have been completed."),
      Button("View Answers", viewanswers),
      Button("Return Home", index)
  ])
```

This also means you don't need a second Calculate Answers button on this page.
Important: The user must enter valid compound information before reaching this route. Your default reaction currently has zero molar masses and coefficients, which would cause division-by-zero errors.

5. viewanswers

Your current Text statement has two problems:

- The values inside parentheses are being treated as ordinary text.
- Your Button is accidentally included inside the text string.

Use an f-string to insert the actual values:

```
@route
def viewanswers(state: State) -> Page:
  return Page(state, [
     Text(f"The limiting reactant is {state.limreac.name}."),
     Text(f"The theoretical yield is {state.theoyield} g."),
     Text(f"The percent yield is {state.percentyield}%."),
     Button("Return Home", index)
  ])
```

This will display the actual calculated results.

6. explainprocess

Your explanation is chemically sound and explains the three major calculations.
You just need to add a navigation button:

```
@route
def explainprocess(state: State) -> Page:
  return Page(state, [
     Text("The way this process is completed is in 3 major steps. First, you must determine which reactant is limiting, meaning runs out first. This is done by converting the mass of each reactant to moles of reactant and then to moles of product. Whichever reactant produces less moles of product is the limiting reactant. Second, to find the theoretical yield, take the moles of product produced by the limiting reactant and convert to mass of product using molar mass. Third, to find percent yield, use the given mass of product, divide it by the theoretical yield, and multiply by 100."),
     Button("Return Home", index)
  ])
```

7. How to correctly collect compound information

I looked at the Drafter input documentation, and here's the important distinction.
Drafter uses TextBox() to collect information. The name you give the TextBox becomes a parameter in the route that receives the submitted information. Drafter can convert the input into a float or int based on the parameter's type annotation.

Drafter Documentation
+1

For example, to collect the name and mass of your first reactant:

```
@route
def compoundinput(state: State) -> Page:
  return Page(state, [
     Text("Enter Reactant 1"),
     Text("Compound Name:"),
     TextBox("reac1_name"),
     Text("Compound Mass:"),
     TextBox("reac1_mass"),
```

```
      Button("Save", savecompoundinput)
  ])
```

You would then need another route to receive and save the information:

```
@route
def savecompoundinput(state: State, reac1_name: str, reac1_mass: float) -> Page:
  state.balancedreaction.reac1.name = reac1_name
  state.balancedreaction.reac1.mass = reac1_mass

  return index(state)
```

This is just a demonstration for two fields. You would eventually collect the molar mass and coefficient too, and repeat the process for the other three compounds.
You can also put all four compounds' input fields on one page and save them together, which I think would be easier for your project.
See the
Drafter Forms and Input documentation
for more examples.

8. Two additional things to watch

A. Your default values
Your default compounds have molar masses and coefficients of zero.
Your calculation functions divide by both of these values, so make sure users have entered valid values before they can calculate.
B. Your home button
You can use:

```
Button("Return Home", index)
```

This preserves the current State and avoids needing to use your separate home route.

Overall assessment

State dataclass
Correct
Default state
Correct
Index and navigation
Minor fixes
Reset
Correct
Compound input
Needs implementation
Calculate answers
Needs implementation
View answers
Needs formatting
Explain process
Almost finished
My recommendation: Work on compoundinput next. Getting the four compounds' information into your Reaction dataclass is the foundation for everything else your website needs to do.

```
@dataclass
```

```
class State:
    message: str
    balancedreaction: Reaction
    limreac: Compound
    theoyield: float
    percentyield: float
#default state
state0 = State("Welcome to Home Page of Limiting Reactant Calculator, choose your action below", Reaction(Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0)), None, 0.0, 0.0)


#route function 1: index
@route
def index(state: State) -> Page:
    """
    This function produces the initial home page of the website, which has 5 buttons on it.
The input values button takes the user to the input values page, where they can input the values for one compound of the reaction.
The calculate answers button takes the user to the calculate answers page, where they can calculate the answers for the reaction that they input.
The view answers button takes the user to the view answers page where the identified limiting reactant, theoretical yield, and percent yield are shown in a text box.
The process explanation button takes the user to the process explanation page where there is a text box explaining how the answers were calculated
    """
    return Page(state, [Text(state.message), Button("Input Values", compoundinput), Button("Calculate Answers", calculateanswers), Button("View Answers", viewanswers), Button("Process Explanation", explainprocess), Button("Reset Values", reset)]


#route function 2: reset
```

```
@route

def reset(state: State) -> Page:

    return Page(state0)


#route function 3: home

@route

def home(state: State) -> Page:

    return Page(state)


#route function 4: compoundinput

@route

def compoundinput(state: State) -> Page:

    """

    Upon clicking the input values button on the home page, the user is taken to the compound input page, where there are 4 buttons, one to input the name of the compound, one to input the mass, one to input the molar mass, and one to input the coefficient in the balanced reaction. After going through each button, the user can press the 5th home button to return to the home page and see an updated balanced reaction with one compound filled in.

    """

    return Page(state, [Text("Go through each input section one time for each compound in the reaction."), Button("Compound Name", TextBox("What is the formula for this compound?"), Button("Compound Mass", TextBox("What is the given, not calculated mass of this compound, in grams?"), Button("Compound Molar Mass", TextBox("What is the molar mass of this compound, in g/mol?"), Button("Compound Coefficient", TextBox("What is the coefficient of this compound in the given balanced reaction?"), Button("Return Home", home)])


#route function 5: calculate answers

@route

def calculateanswers(state: State) -> Page:

    """
```

Upon clicking the calculate answers button on the home page, the user is taken to the calculate answers page, where there are 2 buttons. 1 will calculate the answers, which this function does, and the other returns the user to the home page.

"""

state.limreac = findlimreac(state.balancedreaction)

state.theoyield = findtheoyield(state.balancedreaction)

state.percentyield = findpercentyield(state.balancedreaction)

return Page(state, [Text("Calculations done."), Button("View Answers", viewanswers), Button("Return Home", home)])

#route function 6: view answers

@route

def viewanswers(state: State) -> Page:

"""

Upon clicking the view answers button on the home page, the user is taken to the calculate answers page, where there is a textbox displaying the answers for the specified limiting reactant, theoretical yield, and percent yield. There is also a button that returns to the home page

"""

return Page(state, [Text("The limiting reactant is {state.limreac}, the theoretical yield is {state.theoyield}, and the percent yield is {state.percentyield}"), Button("Return Home", home)])

#route function 7: explain process

@route

def explainprocess(state: State) -> Page:

"""

Upon clicking the process explanation button on the home page, the user is taken to the process explanation page, where there is a textbox that shows how the calculations were completed, and a button for returning home.

"""

return Page(state, [Text("The way this process is completed is in 3 major steps. First, you must determine which reactant is limiting, meaning runs out first. This is done by converting the mass of each reactant to moles of reactant and then to moles of product. Whichever reactant produces less moles of product is the limiting reactant. Second, to find the theoretical yield, take the moles of product produced by the limiting reactant and convert to mass of product using molar mass. Third, to find percent yield, use the given mass of product, divide it by the theoretical yield, and multiply by 100."), Button("Return Home", home)])

start_server(state0)
…
Your updated code is getting closer! The biggest improvement is that you've moved the calculations into the body of calculateanswers, which is exactly where they belong. There are still a few important issues to fix before the website will work.

1. Your State and state0

Your original State definition has five fields:

1. message
2. balancedreaction
3. limreac
4. theoyield
5. percentyield

Your state0 must provide exactly five values, in that order. The edited version has a misplaced None inside the Reaction(...) constructor.
Use this structure:

```
state0 = State(
  "Welcome to Home Page of Limiting Reactant Calculator, choose your action below",
  Reaction(
     Compound("", 0.0, 0.0, 0),
     Compound("", 0.0, 0.0, 0),
     Compound("", 0.0, 0.0, 0),
     Compound("", 0.0, 0.0, 0)
  ),
  None,
  0.0,
  0.0
)
```

You can initialize limreac as None because no calculation has happened yet.

2. Your index route

There are two problems in your edited version:

- The return statement needs to be indented inside index.
- You should use Page(state, ...), not Page(state0, ...).

Using state0 would reset the user's information whenever they return to the home page.

```
@route
def index(state: State) -> Page:
  return Page(state, [
```

```
    Text(state.message),
    Button("Input Values", compoundinput),
    Button("Calculate Answers", calculateanswers),
    Button("View Answers", viewanswers),
    Button("Process Explanation", explainprocess),
    Button("Reset Values", reset)
  ])
```

3. Your compoundinput route

You made the right decision to remove the input() calls, but the TextBox components need to be structured differently.
A TextBox needs a name that connects it to a parameter in another route. For example:
TextBox("reac1_name")
Then, a route can receive that value using a matching parameter:
@route
def savecompoundinput(state: State, reac1_name: str) -> Page:
The name reac1_name must match exactly.
Your current design also has a bigger issue: you need to collect four values for each of four compounds, which means 16 inputs total.
I recommend putting all four compounds on one page, with four sections:

- Reactant 1: name, mass, molar mass, coefficient
- Reactant 2: name, mass, molar mass, coefficient
- Product 1: name, actual mass, molar mass, coefficient
- Product 2: name, mass, molar mass, coefficient

Then have one Save button that sends all the values to a new route, such as savecompoundinput.
For now, don't worry about writing all 16 inputs. First, understand how one input gets saved.

4. Your calculateanswers route

Good structure
This is now structured correctly:

```
@route
def calculateanswers(state: State) -> Page:
  state.limreac = findlimreac(state.balancedreaction)
  state.theoyield = findtheoyield(state.balancedreaction)
  state.percentyield = findpercentyield(state.balancedreaction)

  return Page(state, [
    Text("Calculations done."),
    Button("View Answers", viewanswers),
    Button("Return Home", home)
  ])
```

One important thing: make sure the user has entered valid masses, molar masses, and coefficients before allowing these calculations. Otherwise, you could encounter division by zero.

5. Your viewanswers route

You are very close here. You need an f before the string in Text() to make Python insert the values.

You should also display the limiting reactant's name rather than the entire Compound object.

```
@route
def viewanswers(state: State) -> Page:
  return Page(state, [
      Text(f"The limiting reactant is {state.limreac.name}."),
      Text(f"The theoretical yield is {state.theoyield} grams."),
      Text(f"The percent yield is {state.percentyield}%."),
      Button("Return Home", home)
  ])
```

For example, if the limiting reactant is aluminum, the first line will display:
The limiting reactant is Al.

6. Your explainprocess route

Your chemistry explanation is correct! You have all three major steps:

1. Determine the limiting reactant.
2. Calculate theoretical yield.
3. Calculate percent yield.

Your Return Home button is also correctly structured.

7. Your start_server

Your edited version at the bottom has multiple conflicting State initializations. You only need one.
For now, use:

```
start_server(state0)
```

What I recommend doing next

Your seven routes are a good starting point, but your compoundinput route is the biggest unfinished part.
I would work on getting one compound's four inputs working first. Once you can save its name, mass, molar mass, and coefficient into state.balancedreaction.reac1, you can repeat the same pattern for the other three compounds.
That will also give you a better understanding of how Drafter passes information between routes.

show me how compoundinput and savecompoundinput should look
I recommend using one input page with all four compounds, followed by one Save button. This way, the user enters the entire balanced reaction at once instead of having to visit the input page repeatedly.
Let's start with the four values for each compound.

1. compoundinput route

This route displays the input boxes. Each TextBox has a name that identifies the value it will send to savecompoundinput.

```
@route
def compoundinput(state: State) -> Page:
  return Page(state, [
      Text("Enter Reactant 1"),
```

```
    Text("Compound Name:"),
    TextBox("reac1_name"),
    Text("Mass (g):"),
    TextBox("reac1_mass"),
    Text("Molar Mass (g/mol):"),
    TextBox("reac1_molarmass"),
    Text("Coefficient:"),
    TextBox("reac1_coeff"),

    Text("Enter Reactant 2"),
    Text("Compound Name:"),
    TextBox("reac2_name"),
    Text("Mass (g):"),
    TextBox("reac2_mass"),
    Text("Molar Mass (g/mol):"),
    TextBox("reac2_molarmass"),
    Text("Coefficient:"),
    TextBox("reac2_coeff"),

    Text("Enter Product 1"),
    Text("Compound Name:"),
    TextBox("prod1_name"),
    Text("Actual Mass (g):"),
    TextBox("prod1_mass"),
    Text("Molar Mass (g/mol):"),
    TextBox("prod1_molarmass"),
    Text("Coefficient:"),
    TextBox("prod1_coeff"),

    Text("Enter Product 2"),
    Text("Compound Name:"),
    TextBox("prod2_name"),
```

2. savecompoundinput route

This route receives all 16 values and stores them in your existing Reaction and Compound dataclasses.

```
@route
def savecompoundinput(state: State,
                      reac1_name: str, reac1_mass: float,
                      reac1_molarmass: float, reac1_coeff: int,
                      reac2_name: str, reac2_mass: float,
                      reac2_molarmass: float, reac2_coeff: int,
                      prod1_name: str, prod1_mass: float,
                      prod1_molarmass: float, prod1_coeff: int,
                      prod2_name: str, prod2_mass: float,
                      @dataclass
class State:
  message: str
  balancedreaction: Reaction
  limreac: Compound
```

```
  theoyield: float
  percentyield: float

#default state
state0 = State("Welcome to Home Page of Limiting Reactant Calculator, choose your action
below", Reaction(Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0,
0), Compound("", 0.0, 0.0, 0)), None, 0.0, 0.0)



#route function 1: index
@route
def index(state: State) -> Page:
  """
  This function produces the initial home page of the website, which has 5 buttons on it.

The input values button takes the user to the input values page, where they can input the values
for one compound of the reaction.

The calculate answers button takes the user to the calculate answers page, where they can
calculate the answers for the reaction that they input.

The view answers button takes the user to the view answers page where the identified limiting
reactant, theoretical yield, and percent yield are shown in a text box.

The process explanation button takes the user to the process explanation page where there is a
text box explaining how the answers were calculated
  """
```

3. How the two routes connect

The important part is that the names in TextBox match the parameter names in savecompoundinput.
For example:
In compoundinput:
TextBox("reac1_name")
In savecompoundinput:
reac1_name: str
Stored in your dataclass:
Compound(reac1_name, reac1_mass,
    reac1_molarmass, reac1_coeff)
Drafter automatically passes the entered values to the matching route parameters. The type annotations (str, float, and int) tell Drafter what types to convert the inputs into.
One important note: This example assumes the user enters valid numbers in every numerical box. You should eventually add input validation so empty or invalid values don't cause errors during calculations.
Also, your prod1_mass represents the actual yield, which is what your findpercentyield function uses.
I'd test this with just Reactant 1 first, then add the other three compounds once you confirm the saving process works.

```
@dataclass

class State:

    message: str

    balancedreaction: Reaction

    limreac: Compound

    theoyield: float

    percentyield: float

#default state

state0 = State("Welcome to Home Page of Limiting Reactant Calculator, choose your action below", Reaction(Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0), Compound("", 0.0, 0.0, 0)), None, 0.0, 0.0)



#route function 1: index

@route

def index(state: State) -> Page:

    """

    This function produces the initial home page of the website, which has 5 buttons on it.

The input values button takes the user to the input values page, where they can input the values for one compound of the reaction.

The calculate answers button takes the user to the calculate answers page, where they can calculate the answers for the reaction that they input.

The view answers button takes the user to the view answers page where the identified limiting reactant, theoretical yield, and percent yield are shown in a text box.

The process explanation button takes the user to the process explanation page where there is a text box explaining how the answers were calculated

    """

    return Page(state, [Text(state.message), Button("Input Values", compoundinput), Button("Calculate Answers", calculateanswers), Button("View Answers", viewanswers), Button("Process Explanation", explainprocess), Button("Reset Values", reset)])
```

```
#route function 2: reset

@route

def reset(state: State) -> Page:

    return Page(state0)


#route function 3: home

@route

def home(state: State) -> Page:

    return Page(state)


#route function 4: compoundinput

@route

def compoundinput(state: State) -> Page:

    """

    Upon clicking the input values button on the home page, the user is taken to the compound input page, where there are 4 buttons, one to input the name of the compound, one to input the mass, one to input the molar mass, and one to input the coefficient in the balanced reaction. After going through each button, the user can press the 5th home button to return to the home page and see an updated balanced reaction with one compound filled in.

    """

    return Page(state, [

        Text("Enter Reactant 1"),

        Text("Compound Name:"),

        TextBox("reac1_name"),

        Text("Mass (g):"),

        TextBox("reac1_mass"),

        Text("Molar Mass (g/mol):"),
```

TextBox("reac1_molarmass"),

Text("Coefficient:"),

TextBox("reac1_coeff"),

Text("Enter Reactant 2"),

Text("Compound Name:"),

TextBox("reac2_name"),

Text("Mass (g):"),

TextBox("reac2_mass"),

Text("Molar Mass (g/mol):"),

TextBox("reac2_molarmass"),

Text("Coefficient:"),

TextBox("reac2_coeff"),

Text("Enter Product 1"),

Text("Compound Name:"),

TextBox("prod1_name"),

Text("Actual Mass (g):"),

TextBox("prod1_mass"),

Text("Molar Mass (g/mol):"),

TextBox("prod1_molarmass"),

Text("Coefficient:"),

TextBox("prod1_coeff"),

Text("Enter Product 2"),

Text("Compound Name:"),

TextBox("prod2_name"),

Text("Mass (g):"),

TextBox("prod2_mass"),

```
        Text("Molar Mass (g/mol):"),
        TextBox("prod2_molarmass"),
        Text("Coefficient:"),
        TextBox("prod2_coeff"),
        Button("Save Reaction", savecompoundinput),
        Button("Return Home", home)
    ])

#route function 5: save compound input
@route
def savecompoundinput(state: State,
                      reac1_name: str, reac1_mass: float,
                      reac1_molarmass: float, reac1_coeff: int,
                      reac2_name: str, reac2_mass: float,
                      reac2_molarmass: float, reac2_coeff: int,
                      prod1_name: str, prod1_mass: float,
                      prod1_molarmass: float, prod1_coeff: int,
                      prod2_name: str, prod2_mass: float,
                      prod2_molarmass: float, prod2_coeff: int) -> Page:
    state.balancedreaction = Reaction(
        Compound(reac1_name, reac1_mass, reac1_molarmass, reac1_coeff),
        Compound(reac2_name, reac2_mass, reac2_molarmass, reac2_coeff),
        Compound(prod1_name, prod1_mass, prod1_molarmass, prod1_coeff),
        Compound(prod2_name, prod2_mass, prod2_molarmass, prod2_coeff)
    )
```

```
    return Page(state, [
        Text("Your reaction has been saved!"),
        Button("Return Home", home),
        Button("Input Values Again", compoundinput)
    ])


#route function 6: calculate answers
@route
def calculateanswers(state: State) -> Page:
    """
    Upon clicking the calculate answers button on the home page, the user is taken to the calculate answers page, where there are 2 buttons. 1 will calculate the answers, which this function does, and the other returns the user to the home page.
    """
    state.limreac = findlimreac(state.balancedreaction)
    state.theoyield = findtheoyield(state.balancedreaction)
    state.percentyield = findpercentyield(state.balancedreaction)
    return Page(state, [Text("Calculations done."), Button("View Answers", viewanswers), Button("Return Home", home)])


#route function 7: view answers
@route
def viewanswers(state: State) -> Page:
    """
    Upon clicking the view answers button on the home page, the user is taken to the calculate answers page, where there is a textbox displaying the answers for the specified limiting reactant, theoretical yield, and percent yield. There is also a button that returns to the home page
    """
```

```
    return Page(state, [Text(f"The limiting reactant is {state.limreac.name}, the theoretical yield is {state.theoyield}, and the percent yield is {state.percentyield}"), Button("Return Home", home)])



#route function 8: explain process

@route

def explainprocess(state: State) -> Page:

    """

    Upon clicking the process explanation button on the home page, the user is taken to the process explanation page, where there is a textbox that shows how the calculations were completed, and a button for returning home.

    """

    return Page(state, [Text("The way this process is completed is in 3 major steps. First, you must determine which reactant is limiting, meaning runs out first. This is done by converting the mass of each reactant to moles of reactant and then to moles of product. Whichever reactant produces less moles of product is the limiting reactant. Second, to find the theoretical yield, take the moles of product produced by the limiting reactant and convert to mass of product using molar mass. Third, to find percent yield, use the given mass of product, divide it by the theoretical yield, and multiply by 100."), Button("Return Home", home)])



start_server(state0)
```
…
This is a significant improvement! You've now added savecompoundinput, connected all 16 input boxes to it, and reached 8 route functions, so you have exceeded your assignment's minimum of 7.
Most of your code is structured correctly. I see a few things worth fixing or considering.

1. Your compoundinput route looks correct

Correct structure
You have correctly:

- Separated the four compounds.
- Given each input box a unique name.
- Connected the Save Reaction button to savecompoundinput.
- Included a Return Home button.

One small improvement: your docstring still describes four buttons for entering one compound at a time. Your new design actually uses one page with 16 input boxes. Update the docstring to describe what the route does now.
```
```

2. Your savecompoundinput route looks correct

Correct structure
You correctly match the TextBox names to the route parameters.
For example:
TextBox("reac1_name")
matches:
reac1_name: str
And you correctly reconstruct your Reaction using four Compound objects:

```
state.balancedreaction = Reaction(
  Compound(reac1_name, reac1_mass, reac1_molarmass, reac1_coeff),
  Compound(reac2_name, reac2_mass, reac2_molarmass, reac2_coeff),
  Compound(prod1_name, prod1_mass, prod1_molarmass, prod1_coeff),
  Compound(prod2_name, prod2_mass, prod2_molarmass, prod2_coeff)
)
```

This is exactly how you should update your existing reaction.

3. One thing to change in calculateanswers

Currently, your route calculates all three answers immediately when the user clicks Calculate Answers.
That is fine, but you should make sure the user has saved their reaction first. Otherwise, your calculations will attempt to divide by zero because your default molar masses and coefficients are zero.
For now, you could add a basic check before the calculations:

```
if state.balancedreaction.reac1.molarmass == 0:
  return Page(state, [
     Text("Please enter and save your reaction first."),
     Button("Input Values", compoundinput),
     Button("Return Home", home)
  ])
```

This checks the first reactant's molar mass. Eventually, you should check all four compounds' numerical values.

4. Your viewanswers route

This is now correct:

```
return Page(state, [
  Text(f"The limiting reactant is {state.limreac.name}, the theoretical yield is {state.theoyield}, and the percent yield is {state.percentyield}"),
  Button("Return Home", home)
])
```

One possible improvement is to add units and round the results:

```
Text(f"The theoretical yield is {round(state.theoyield, 2)} g.")
Text(f"The percent yield is {round(state.percentyield, 2)}%.")
```

5. Your route count

Route	Purpose

index	Main page
reset	Reset all values
home	Return home
compoundinput	Display input boxes
savecompoundinput	Save the reaction
calculateanswers	Calculate results
viewanswers	Display results
explainprocess	Explain calculations

8 routes

6. One important thing to test

Before moving on, test the following sequence:

1. Open the website.
2. Click Input Values.
3. Enter a reaction, such as 2Al + Fe2O3 -> Al2O3 + 2Fe.
4. Save the reaction.
5. Return home.
6. Click Calculate Answers.
7. Click View Answers.
8. Confirm that the limiting reactant and yields are displayed.

My recommendation: Your next priority should be testing whether Drafter successfully passes all 16 inputs into savecompoundinput. If that works, the main website structure is in good shape.

here is the full code. will there be any syntactical errors
I reviewed the full file. There are a few things to fix, but the overall structure of your code is sound. I also checked whether the code compiles after correcting the formatting characters in the pasted file.
The most important distinction is that some issues are actual syntax errors, while others will cause problems when the website runs.

1. Syntax issues to fix

A. The import statement
Your file shows:
from drafter import *
It should be:
from drafter import *
B. The quotation marks in reaction1 and its test
Your file uses curly quotation marks:
Compound(“Al”, 124.0, 27.0, 2)

Python requires straight quotation marks:
Compound("Al", 124.0, 27.0, 2)
Make this correction everywhere you use those curly quotation marks, including your assert_equal test.
C. The comment markers
Your pasted file displays some comment markers as \#. In your actual Python code, comments should start with #, without a backslash.
These may be formatting escapes from the pasted Markdown, but make sure they are not present in your Python editor.

2. Test errors (not syntax errors)

Your helper functions are structured correctly, but two of your tests will fail because the expected answers are rounded while your functions return more decimal places.
Your current tests are:
assert_equal(findtheoyield(reaction1), 234.2)
assert_equal(findpercentyield(reaction1), 99.99)
The actual calculated values are approximately:

- Theoretical yield: 234.2222...
- Percent yield: 99.9905...

Change the tests to:
assert_equal(round(findtheoyield(reaction1), 1), 234.2)
assert_equal(round(findpercentyield(reaction1), 2), 99.99)
This keeps your actual calculation functions unchanged.

3. Problems that could occur when running the website

These aren't syntax errors, but they are important.
A. Your home route displays a blank page.
Currently:
```
@route
def home(state: State) -> Page:
  return Page(state)
```
Change it to:
```
@route
def home(state: State) -> Page:
  return index(state)
```
This makes your Return Home buttons display the actual homepage.
B. Your calculation route can divide by zero.
If someone clicks Calculate Answers before saving a reaction, the default molar masses and coefficients are zero.
You should check that the user has entered valid values before running:
state.limreac = findlimreac(state.balancedreaction)
Otherwise, Python can produce a ZeroDivisionError.
C. Your limreac type annotation
You currently have:
limreac: Compound
But your default state uses:
None

This isn't a syntax error, but the annotation doesn't accurately describe the initial value. If your class has covered optional types, you could use Compound | None. Otherwise, you can leave it for now.

4. Overall review

Section	Result
Dataclasses	Correct
Mass-to-mole function	Correct
Mole-to-mole function	Correct
Limiting reactant function	Correct
Theoretical yield function	Correct
Percent yield function	Correct
State and default state	Valid structure
Eight route functions	Valid structure
Input and save routes	Valid structure
Calculation and answer routes	Need safeguards

My conclusion: After correcting the import formatting and quotation marks, the code compiles syntactically. The two rounded tests need adjustment, and the home route and calculation safeguards need attention before you can expect the website to work correctly.
One limitation: a successful syntax check doesn't guarantee that Drafter will accept every route or that the website will behave correctly at runtime. I would test the input and save process next.

find any errors and find a spot to put a for loop in the code
Worked for 32s
I reviewed your full code. Overall, your helper functions and the structure of your eight Drafter routes are on the right track. I found a few things to fix, plus a good place to add a for loop.

1. Errors and things to fix

A. Your two yield tests may fail

Your calculations produce decimal values that are slightly different from the rounded values in your tests.
For example, your theoretical yield is approximately 234.2222, not exactly 234.2. Your percent yield is approximately 99.9905, not exactly 99.99.
Pasted markdown(2).md
Change the tests to:

```
assert_equal(round(findtheoyield(reaction1), 1), 234.2)

assert_equal(round(findpercentyield(reaction1), 2), 99.99)
```

B. Your calculateanswers route needs an else

In the latest version of your code, calculations happen immediately when the user clicks the button. If they haven't entered a reaction, the molar masses and coefficients are zero, which can cause a division-by-zero error.
You can add an if statement and an else:

```
@route
def calculateanswers(state: State) -> Page:
  if state.balancedreaction.reac1.mass != 0:
    state.limreac = findlimreac(state.balancedreaction)
    state.theoyield = findtheoyield(state.balancedreaction)
    state.percentyield = findpercentyield(state.balancedreaction)

    return Page(state, [
      Text("Calculations done."),
      Button("View Answers", viewanswers),
      Button("Return Home", home)
    ])
  else:
    return Page(state, [
      Text("Please input your reaction first."),
      Button("Input Values", compoundinput),
      Button("Return Home", home)
    ])
```

This checks whether the first reactant has been entered. Eventually, you should also check that the other required masses, molar masses, and coefficients are valid.

C. Your viewanswers route needs a check

If someone clicks View Answers before clicking Calculate Answers, state.limreac is still None. Therefore, state.limreac.name will cause an error.
Add a check:

```
@route
def viewanswers(state: State) -> Page:
  if state.limreac != None:
    return Page(state, [
      Text(f"The limiting reactant is {state.limreac.name}, the theoretical yield is
{state.theoyield} g product, and the percent yield is {state.percentyield}%."),
      Button("Return Home", home)
    ])
  else:
    return Page(state, [
      Text("Please calculate the answers first."),
      Button("Return Home", home)
    ])
```

D. Your home route

In the latest edited code block, your home route returns Page(state). That may display a blank page.
Use:
@route
def home(state: State) -> Page:
  return index(state)
This takes the user back to your main page.

E. Your findlimreac documentation

Your docstring says that findlimreac returns the reactant's name as a string, but your function actually returns the entire Compound object.
Pasted markdown(2).md
Change the Returns section of the docstring to something like:
Returns: Compound - the Compound dataclass representing the limiting reactant

F. One possible formatting issue

The uploaded file displays some characters as * and \#. If those backslashes are actually in your Python editor, remove them:

- from drafter import *
- # helper function section
- molesoutput = molesinput * coeffoutput / coeffinput

The backslashes should not be there in actual Python code.
I also noticed that the latest edited code snippet appears to have part of savecompoundinput interrupted by the State class, with leftover parameters after start_server(state0). If that is how your actual .py file looks, fix that structure. The complete function in your uploaded file is correctly arranged.

2. Where to put a for loop

I recommend putting a for loop inside your compoundinput route.
Right now, you repeat the same pattern four times: Reactant 1, Reactant 2, Product 1, and Product 2.
A for loop can create those four sections for you. This is a good use of a loop because you're repeating the same instructions with different names.
Replace your current compoundinput route with:
@routeUnusable Value
The content you returned in this Page was a State, but it needs to be text, a number, a boolean, a component, or a list of those.

request.route_execution_failed: Error while processing request for URL 'reset': The content of a Page must be a list of strings, numbers, booleans, or components. Found State instead.
def compoundinput(state: State) -> Page:

```
  page_content = []

  compounds = [
     ("Reactant 1", "reac1", "Mass (g):"),
     ("Reactant 2", "reac2", "Mass (g):"),
```

```
    ("Product 1", "prod1", "Actual Mass (g):"),
    ("Product 2", "prod2", "Mass (g):")
]

for title, prefix, mass_label in compounds:
    page_content.append(Text(title))

    page_content.append(Text("Compound Name:"))
    page_content.append(TextBox(prefix + "_name"))

    page_content.append(Text(mass_label))
    page_content.append(TextBox(prefix + "_mass"))

    page_content.append(Text("Molar Mass (g/mol):"))
    page_content.append(TextBox(prefix + "_molarmass"))

    page_content.append(Text("Coefficient:"))
    page_content.append(TextBox(prefix + "_coeff"))

page_content.append(Button("Save Reaction", savecompoundinput))
page_content.append(Button("Return Home", home))

return Page(state, page_content)
```

How the loop works

The compounds list contains four sets of information:

Title	Prefix
Reactant 1	reac1
Reactant 2	reac2
Product 1	prod1
Product 2	prod2

Each time the loop runs, it takes the next set of information and creates the corresponding text and text boxes.
For example, when prefix is "reac1", this line:
TextBox(prefix + "_name")
becomes:
TextBox("reac1_name")
That means you do not need to change the parameters in your savecompoundinput function. The TextBox names still match them.

3. Overall review

Part of your code	Status

Compound and Reaction dataclasses	Good
masstomole	Good
moletomole	Good
findlimreac	Good
findtheoyield	Good
findpercentyield	Good
savecompoundinput	Good in uploaded file
calculateanswers	Needs validation
viewanswers	Needs a check
home	Check latest version
compoundinput	Good place for a loop

My recommendation: Use the for loop in compoundinput. It makes your code shorter, avoids repeating the same instructions, and gives you a clear example of using iteration in your project.

Unusable Value

The content you returned in this Page was a State, but it needs to be text, a number, a boolean, a component, or a list of those.
request.route_execution_failed: Error while processing request for URL 'reset': The content of a Page must be a list of strings, numbers, booleans, or components. Found State instead.
it is giving me this error regarding the reset function. what do I do?
The problem is in your reset function. You're returning Page(state0), but Drafter expects the second argument of Page to be a list of components to display, not a State object.
Your current function is probably:

```
None
@routedef reset(state: State) -> Page:    return Page(state0)
```

Fix

Change it to:

```
None
@routedef reset(state: State) -> Page:    return Page(state0, [        Text(state0.message),
Button("Input Values", compoundinput),        Button("Calculate Answers",
```

```
calculateanswers),        Button("View Answers", viewanswers),        Button("Process Explanation", explainprocess),        Button("Reset Values", reset)    ])
```

Why this works

The Page function needs two things:

1. state0 — the state you want to reset the website to.
2. A list of components — the text and buttons you want displayed.

This resets the stored values to their defaults and displays your home page again.
One important detail: Your index route already creates this exact home page. You can avoid repeating the buttons by writing:

```
None
@routedef reset(state: State) -> Page:    return index(state0)
```

My recommendation
Use the second version. It's shorter and reuses your existing index route.

it doesn't actually reset the values. when I have values already input, and i press reset and then press calculate, it still gives me the answers for the values inputted before resetting
I see the issue. The problem is that return index(state0) displays the default state, but your Drafter application may still be retaining the previously modified state when you navigate between routes.
Let's make the reset function explicitly replace the values in the current state before returning to the home page.

1. Change your reset function

Try this:

```
@route
def reset(state: State) -> Page:
   state.message = state0.message
   state.balancedreaction = state0.balancedreaction
   state.limreac = None
   state.theoyield = 0.0
   state.percentyield = 0.0

   return index(state)
```

This resets all five fields in your State dataclass:

State variable	Reset value
message	Welcome message
balancedreaction	Empty reaction
limreac	None

theoyield	0.0
percentyield	0.0

2. Why this should help

Your savecompoundinput function changes:
state.balancedreaction
Your calculateanswers function changes:
state.limreac
state.theoyield
state.percentyield
The reset function now explicitly restores all of those values.

3. Test it

1. Input your aluminum and iron oxide reaction.
2. Save the reaction.
3. Calculate the answers.
4. Press Reset Values.
5. Press Calculate Answers again.

You should now get the message asking you to input your reaction first.
One additional note: This assumes your Drafter version preserves state mutations in the way your other routes currently use them. If it still retains old values, we should look at how your buttons and routes are passing state.